Paras: handbook

Labels, dataset and the neural network (CNN)

Read 00_Start_Here first; then this handbook.

Machine Learning (702DB0C015), B.Tech CSE (Data Science), Semester V, 2026-27

Contents

1. Part brief
2. Knowledge pack
3. Presentation guide
4. Viva questions and answers
5. Diagrams

Paras: labels, dataset building and the neural network (CNN)

Marks this part carries: Model Development (4, shared with Satvik) and the model-ready data part of Data Preprocessing (3, shared with Aaryan). **Why it matters:** this is the deep-learning part of the project, and the part where "garbage labels in, garbage model out" is most visible. It also contains the best data-centric story: the model found a labelling error.

Your four jobs

1. **Where labels come from:** injection-recovery (planted fakes), known asteroids from catalogues, and "everything else is bogus", with the label-noise caveats and the quality checks.
2. **Building the dataset:** cutting 3 x 31 x 31 stamps, normalising, augmenting, balancing classes, assembling train/validation sets without leakage.
3. **The neural network:** from "a neuron" to backpropagation (the from-scratch notebook), then the real CNN: architecture, training loop, regularisation, results versus the baseline.
4. **Error analysis and cross-fitting:** what the model's most confident mistakes revealed, and how its scores are produced without leakage for the next stage.

What is NOT yours

Downloads and the first detection step: Aaryan. Linking, tracklet features, trees and the metric definitions: Satvik. Scoreboard, deep search and viewer: Advik. You hand Satvik a `detections` table with a `score` column and a threshold.

Where your code is (in `project/`)

Area	Files	What they do
Simulate	`src/asthunt/simulate/render_point_source.py`, `plan_injections.py`, `inject_point_source.py`, `plant_fakes_in_exposure.py` (+ trailed versions)	draw a fake from ZTF's own PSF, choose brightness/speed/position, add it to science and difference
Label	`src/asthunt/label/match_detections_to_positions.py` (KD-tree), `query_known_asteroids.py`, `query_horizons_positions.py`, `refine_known_positions.py`, `load_known_asteroids.py`, `label_detections.py`	decide real or bogus for each blob
Dataset	`src/asthunt/dataset/build_night_detections.py`, `normalize_stamps.py`, `augment_stamps.py`, `balance_classes.py`, `assemble_stamp_dataset.py`, `relabel_night_detections.py`, `make_unit_tag.py`, `make_unit_rng.py`	one night -> detections + stamps; model-ready arrays

Area	Files	What they do
Stamps	`src/asthunt/detect/cut_triplet_stamps.py`	31x31 science / reference / difference cut-outs
Models	`src/asthunt/models/define_realbogus_cnn.py`, `train_realbogus_cnn.py`, `predict_realbogus.py`, `cross_fit_realbogus_scores.py`	the CNN: define, train, score, cross-fit
Realism check	`src/asthunt/evaluate/measure_injection_realism.py`	adversarial validation of fakes vs real asteroids
Config	`configs/inject.yaml`, `dataset.yaml`, `train.yaml`	all the settings
Tests	`tests/simulate`, `tests/label`, `tests/dataset`, `tests/models`	includes a tiny CNN that must learn "blob in the centre = real"
Lessons and notebooks	`docs/injection_labelling.md`, `label_quality.md`, `dataset_building.md`, `neural_networks.md`, `realbogus_cnn.md`; notebooks with the same names	

What you hand over

- The model and its numbers for the report: architecture, training recipe, results versus baseline, error analysis, cross-fitting.
- Figures: the triplet picture, the CNN architecture, the training/validation curves, a few correct and wrong predictions (from the notebook).
- A clear statement of label sources and their weaknesses (this is critical for the evaluation chapter).

Your seven key messages

1. **The sky is not labelled, so we make labels** (planted fakes + known asteroids + "the rest"), and we say honestly where each one is weak.
2. **A real asteroid shows in science and difference but not in the reference**, which is why the CNN gets all three pictures at once.
3. **A small CNN is enough:** 31x31 stamps, local patterns, 72,145 weights; fast on a laptop GPU and hard to overfit.
4. **We judge by validation average precision on realistic (imbalanced) data, not by training loss.** Training is balanced and augmented; validation keeps the true 1:33.
5. **CNN vs baseline:** AP 0.899 vs 0.780; at 90% recall false alarms fall from 813 to 176 per frame (4.6x); on the untouched test night AP 0.938.
6. **The CNN found a labelling bug:** its most confident "bogus" detections were known asteroids; the catalogue positions (SkyBoT) were about 2.9 arcsec off. Fixed by using JPL Horizons.
7. **Cross-fitting** stops the next model from learning over-confident scores.

Numbers you must know by heart

Number	Meaning	Source
31 x 31 x 3	stamp size and channels	`configs/detect.yaml`
72,145	learnable weights in the CNN	`docs/realbogus_cnn.md` (derivation in your knowledge pack)
31 -> 15 -> 7 -> 3	spatial size after the three max-pools	`define_realbogus_cnn.py`
0.001 / 256 / 25 / 4	learning rate / batch / max epochs / patience	`configs/train.yaml`
0.3	dropout	`define_realbogus_cnn.py`
0.780 -> 0.899 -> 0.938	AP: baseline (val), CNN (val), CNN (test night)	`docs/realbogus_cnn.md`
813 -> 176	false positives per frame at 90% recall	same
1,059 : 36,161	real : bogus detections on one training night (1 : 34)	`docs/injection_labelling.md`
8,863 real + 5 bogus per real = 53,178; 16.7% real	the balanced training set	`docs/dataset_building.md`
67,041; 3.2% real	the realistic validation set	same
2.72 px -> 0.26 px	SkyBoT vs Horizons median position error	`docs/label_quality.md`
0.46-0.48	AUC of "fake vs real" classifier (0.5 = indistinguishable)	same
1.15% and 1.3	chance of a random point landing within 2 px of a detection; expected chance matches per frame	`docs/injection_labelling.md`

Hands-on tasks (do these yourself)

1. `cd project && pytest tests/simulate tests/label tests/dataset tests/models -q`.
2. Re-run `notebooks/neural_networks.ipynb` end to end; reproduce the gradient check; change the learning rate to 5.0 and describe what you see.
3. Re-run `notebooks/realbogus_cnn.ipynb` (about 7 minutes on an M3); look at the 12 most confident mistakes yourself and say what each one is.
4. Do the parameter-count derivation by hand (knowledge pack, section 4.2) and check it equals 72,145.
5. Change `dropout` from 0.3 to 0.0 in a copy and predict (then observe) the effect on the gap between training and validation.

Pitfalls

- Saying "the CNN is 99% accurate": accuracy is meaningless at 3% positives; use AP and false alarms at fixed recall.
- Forgetting that training is balanced (16.7% real) while the real sky is about 3%: raw probabilities are inflated.

- Calling labels "ground truth": they are "best available", with known noise.
- Calling `.numpy()` on a tensor with gradients, or forgetting `model.eval()` when scoring (dropout must be off).
- Comparing models at different thresholds instead of at fixed recall.

Paras: knowledge pack

0. Read first (in this order)

`00_Start_Here/01_Project_Overview` -> `02_Domain_Primer` (sections 3-6, 11) -> `03_ML_Primer` (sections A, B, C) -> then `project/docs/ml_basics.md` (logistic regression, features, labels), `injection_labelling.md`, `label_quality.md`, `dataset_building.md`, `neural_networks.md`, `realbogus_cnn.md`.

---

1. Labels: where they come from and how good they are

There is no list of "all asteroids in this image". There is a list of things we put there ourselves, and a catalogue of things others found.

1.1 Injection-recovery (planted fakes) -> perfect labels

- `plan_injections`: for each of 300 fakes per night: magnitude uniform 17-22, speed uniform 5-100 arcsec/hour, random direction, random start; positions follow the **real frame times** on **frame 1's sky grid**, then are converted through the sky to each frame's own pixels (after we found pointing shifts ignored the old way).
- `render_point_source`: the fake is drawn from ZTF's **own PSF** of that frame (not a textbook Moffat), shifted by a sub-pixel amount with cubic interpolation and rescaled so the total flux is exact (otherwise every fake would sit on a pixel centre and a model could learn "perfectly centred = fake").
- `inject_point_source`: Poisson noise (`Poisson(stamp x gain) / gain`), added to **both science and difference**, so `reference = science − difference` is unchanged: exactly the situation of a new object.
- **Weakness:** fakes are only as realistic as the simulation. Any difference is a distribution shift the model may exploit. So: real PSF, real noise, real times, correct sky motion, and a realism test (1.5).
- Magnitudes are drawn **uniformly**, not like real asteroids (which are mostly faint), so every brightness bin has enough fakes to measure completeness. Remember this when quoting rates.

1.2 Known asteroids -> real labels

Positions of catalogued asteroids at each frame time. We match detections to positions within 2 px with a **KD-tree** (the nearest neighbour in about log n steps instead of comparing 7,700 x 300 pairs per frame). We use only known asteroids brighter than V = 20.5: fainter ones are mostly **chance matches**. The check: P(random point lands within 2 px of some detection) = detections x pi x 2^2 / good-pixel area = 1.15%; 114 known asteroids x 1.15% = 1.3 expected chance matches per frame, which is what we saw, so the early faint "matches" were mostly wrong. **Observer location matters:** SkyBoT defaults to the centre of the Earth; ZTF is about 6,400 km away, shifting an asteroid 1 AU away by up to about 9 arcsec (4 pixels). We pass `location="I41"` (ZTF's observatory code).

1.3 Everything else = bogus

Includes variable stars, artifacts, and the rare real object nobody has catalogued: **label noise** we accept. Consequence: a "false positive" on real data may be a real discovery, which is exactly what the project hunts. **Class imbalance:** one training night: 1,059 real vs 36,161 bogus (1 : 34).

1.4 The label-quality story (two bugs, both found by looking at where the model disagreed with the labels)

1. **SkyBoT offset.** The CNN's most confident "bogus" detections were clean new point sources and **every one was a known asteroid** (V 17-20). SkyBoT's predictions sat about 2.9 arcsec from our detections *in a constant direction* (-2.4, +1.5 px) on every night, with scatter of only 0.1-0.5 px. Our WCS matches Gaia stars to 0.04 px and JPL Horizons matches our detections to 0.1-0.3 arcsec, so the error is in SkyBoT. First fix: a 4-px match radius (AP 0.877 -> 0.894). Better fix (label-quality phase): use **JPL Horizons** positions where available (median error 0.26 px vs 2.72 px, 314 matches over 12 nights) and a 2-px radius (4x less area, 4x fewer chance matches); SkyBoT only as fallback, tagged by source so a detected faint known asteroid is never mistaken for a discovery.
2. **Fakes moved straight in pixel coordinates**, ignoring the telescope's pointing shifts, so real asteroids failed to link while fakes did. Fix: plan fakes on frame 1's sky grid and convert. **Lessons:** check the *direction* of offsets, not only the size (a systematic error hid behind chance coincidences); a model's most confident "mistakes" are the cheapest way to find label errors; test your simulation against real objects, not only against itself.

1.5 Do the fakes look real? (adversarial validation)

Train a classifier to separate fakes from real known asteroids. AUC 0.5 = cannot. Rules: match the confounders (sample fakes with the same magnitude distribution as the real asteroids) and cross-validate by night. Result: AUC 0.46-0.48 (two seeds: 0.481 and 0.463; report 0.47 +/- 0.02, not three decimals of false precision). The same trick detects train/test distribution shift in general.

---

2. Building the dataset

`build_night_detections` per unit: load frames -> plant fakes -> detect -> label -> describe (RA/Dec, common-grid x/y, magnitude, seeing) -> cut stamps -> save `detections__<unit>.parquet`, `stamps__<unit>.npy` (float16), `injections__<unit>.parquet`. Built once, re-used many times (about 40 s per night).

- **Save everything, choose later:** every detection's stamp is kept (about 230 MB per night as float16); balancing happens when assembling, so the same files serve training (balanced), validation (realistic) and the real search (all).
- **Seeds:** each night's generator is seeded with `[project seed, night]`: repeatable (an early draft used Python's `hash`, which changes every run).
- **A float16 bug:** saturated star cores exceed 65,504 (float16's maximum) and became `inf`; fix: clip before converting.
- **Sizes:** 11 nights; per night 32-41k detections, 1,053-1,199 real, 27-37 bogus per real.

2.1 Stamps (the CNN's input)

Three channels of 31 x 31 pixels around the blob: science, reference, difference. Why three? A real new object: blob in science and difference, nothing in reference. Bad subtraction: bright star in both science and reference, dipole in difference. Cosmic ray: sharp spike in science and difference, narrower than a star. Noise: faint speck. Masked edge: step in all. A person compares panels; the CNN learns to. 31 px because the blur is about 2 px wide and 15 px of surroundings is enough to see a neighbouring star or an edge (ZTF's own classifier uses 63 x 63).

2.2 Normalisation, augmentation, balancing

- **Normalise** per stamp and channel: `(value − median) / (1.4826 x MAD)`, clipped to [-10, 50]. Median (not mean) so a star barely moves it; MAD x 1.4826 equals the standard deviation for Gaussian noise but ignores outliers; "5" now means 5 sigma on *any* night; clipping stops a saturated star dominating. Nothing is fitted on training data, so there is no leakage here (contrast `StandardScaler`).
- **Augment** (training only): rotations by 0/90/180/270 degrees and optional mirror = 8 versions of every stamp; the sky has no "up". Acts like more data, teaches orientation does not matter. *Never augment validation or test.*
- **Balance** training: all real rows + 5 bogus per real (8,863 real -> 53,178 stamps, 16.7% real) because with 3% positives the gradient is dominated by easy bogus examples and "always bogus" is a comfortable optimum. Cost: inflated probabilities (the model believes real is 16.7% common). Validation stays realistic (67,041 detections, 3.2% real) so precision and false-alarm numbers are honest.

---

3. Neural-network theory (the from-scratch notebook)

- **Neuron:** `output = activation(w.x + b)`; with a sigmoid it is logistic regression.
- **Why stack:** with ReLU (`max(0,z)`) each hidden neuron contributes a "bent line"; 16 bent lines make a curved boundary (the half-moons: logistic regression 0.88, network 0.94). Without a non-linear activation, stacked layers collapse to one linear function.
- **Forward pass:** `z1 = X W1 + b1; h = ReLU(z1); z2 = h W2 + b2; p = sigmoid(z2)`.
- **Loss:** binary cross-entropy `L = −mean(y log p + (1−y) log(1−p))`. Confident and wrong costs a lot (-log 0.01 = 4.6); confident and right costs almost nothing (-log 0.99 = 0.01).
- **Backpropagation** = chain rule from the output back. Output: for sigmoid + cross-entropy, `dL/dz2 = p − y` (the sigmoid's slope cancels the log's slope). Output weights: `dL/dW2 = h^T (p − y) / n`. Back into the hidden layer: `error_hidden = (p − y) W2^T x ReLU'(z1)` where ReLU' is 1 if z1 > 0 else 0. Hidden weights: `dL/dW1 = X^T error_hidden / n`. Each layer reuses the one after it: one backward sweep gives every gradient.
- **Gradient check:** nudge one weight by epsilon = 1e-5 on both sides: `(L(w+e) − L(w−e)) / 2e` must equal the backprop gradient (ours: -0.02896520 both).
- **Gradient descent:** `w <− w − lr x gradient`. Too small: slow; too large (try 5.0): overshoots and diverges. **Adam** adapts each weight's step using its recent gradients.
- **Initialisation:** small and random; if all weights started equal every neuron would stay identical.
- **PyTorch's four steps:** `optimizer.zero_grad()`, `loss = loss_fn(model(X), y)`, `loss.backward()`, `optimizer.step()`. `BCEWithLogitsLoss` fuses sigmoid and cross-entropy (stable), so the model outputs raw **logits**; `torch.no_grad()` and `model.eval()` when scoring.

- **Overfitting demo:** a 256 x 256 network on 40 points: training loss falls toward 1e-5 (memorising noise) while validation loss is lowest at step 22 then climbs: hence **early stopping**.

---

4. The CNN in detail

4.1 Architecture (`define_realbogus_cnn.py` ; see `07_Diagrams/03_cnn_architecture.svg`)

Input (3, 31, 31) -> [Conv3x3 16 + ReLU] -> [Conv3x3 16 + ReLU] -> MaxPool (31->15) -> [Conv3x3 32 + ReLU] -> [Conv3x3 32 + ReLU] -> MaxPool (15->7) -> [Conv3x3 64 + ReLU] -> MaxPool (7->3) -> Flatten (64 x 3 x 3 = 576) -> Dropout 0.3 -> Dense 64 + ReLU -> Dense 1 -> (sigmoid at scoring time).

- **Convolution:** a 3x3 filter slides over the stamp; at each position it multiplies 9 pixels by 9 learned weights and sums. With 3 input channels each filter has 3x3x3 weights and sees science, reference and difference together, which is how it learns "bright in science and difference, nothing in reference". Padding 1 keeps the size.
- **Pooling:** max-pool halves the size, so deeper filters see larger areas (edges and blobs first, then "point source", "dipole", "streak").
- **Dropout 0.3:** randomly switches off 30% of neurons while training so no neuron is relied on.

4.2 Counting the 72,145 weights (do this yourself)

`params = (in_channels x 3 x 3 + 1) x out_channels` for a conv layer: conv1 (3->16) = 448; conv2 (16->16) = 2,320; conv3 (16->32) = 4,640; conv4 (32->32) = 9,248; conv5 (32->64) = 18,496; dense (576 -> 64) = 576 x 64 + 64 = 36,928; output (64 -> 1) = 65. Sum = 72,145.

4.3 Training recipe (`train_realbogus_cnn.py` , `configs/train.yaml`)

setting	value	why
loss	`BCEWithLogitsLoss`	stable cross-entropy
optimiser	Adam, lr 0.001	robust default
batch	256	stable gradients, fits easily
augmentation	random 90-degree rotations + mirror	sky has no up
early stopping	patience 4 on validation AP	keep the best epoch
Judged by **validation AP on realistic data**, not training loss (training loss 0.08 vs validation 0.035 can both be right: training is balanced and augmented, validation is easier). The final run used all 25 epochs; best validation AP (0.899) at epoch 21.		

4.4 Results (validation, 67,041 detections, 3.2% real)

model	AP	precision at 90% recall	false positives per frame
logistic regression, 6 features	0.780	0.191	813
CNN	**0.899**	**0.522**	**176**
At the same completeness the CNN cuts false alarms **4.6x**; on the untouched test night AP **0.938**. The completeness-vs-magnitude curves look alike at 90% recall because we fixed recall: the CNN's gain is **purity**, which makes linking cheap: after the CNN filter only about 24% of detections remain, and seed pairs drop from 52k to 3.5k.			

4.5 Cross-fitting (stacking without leakage; `07_Diagrams/05_cross_fitting.svg`)

8 training nights -> 4 folds of 2 nights. For each fold train a CNN on the other 6 nights and score the 2 held-out nights; validation and test nights are scored by the final CNN. Without it, the tracklet model would see over-confident CNN scores on training nights and learn "score_min is always high for real things", which would not hold on new nights.

---

5. Cross-part cheat sheet

- **Aaryan:** ZTF difference images; split by night; masks; noise via MAD; blobs by SEP at 2 sigma (7,800 per frame, 173 with SNR > 5); leakage audit (100% -> 50% -> 99%).
- **Satvik:** linking by seed-and-predict with KD-trees on the common sky grid; 17 features (11 + 2 static-neighbour + 4 forced-photometry); XGBoost (300 trees, depth 3, lr 0.05); metrics and thresholds; models tie in cross-validation.
- **Advik:** one deployed path; scoreboard on planted fakes and held-out nights; depth 20.5 -> 21.4 -> 22.1; deep search; live viewer; more data did not help.

6. Self-quiz (answers in `04_Viva_QA.md`)

1. Why does a 3-channel filter help? 2. Training loss is 0.08 and validation 0.035: why can validation be lower? 3. Why do the CNN and baseline completeness curves look alike at 90% recall? 4. Explain cross-fitting in two sentences.
2. How did the CNN reveal the SkyBoT offset? 6. Count the parameters of the first conv layer.
3. Why `BCEWithLogitsLoss`? 8. Why not augment validation data? 9. Why do we add the fake to both science and difference images? 10. Why are fake magnitudes drawn uniformly?

Paras: presentation guide

Your slot: 5 minutes, 5 slides. (In the 12-minute plan: 3 minutes, slides 2, 3 and 5.) Pictures are in `07_Diagrams/`.

Slide 1: "Where do the labels come from?" (1 min)

On the slide: `label_sources.svg`. **Say:** "The sky does not come labelled, so we make labels. First, we plant fake asteroids into the real images with known brightness and speed, using the real blur shape of each frame, so we have perfect labels. Second, we look up known asteroids in the catalogue, which gives us real labels, using positions from NASA's JPL Horizons because they are ten times more accurate than the other service we tried. Third, everything else is called bogus, which means a little label noise, since it includes the rare real object nobody has catalogued. On one training night that gives 1,059 real detections and 36,161 bogus: one to thirty-four."

Slide 2: "Three pictures per blob" (1 min)

On the slide: `triplet_stamp.svg`. **Say:** "For every blob we cut a 31 by 31 pixel stamp from three images: science, reference and difference. A real new object shows up in science and difference but not in the reference. A badly subtracted bright star is bright in science and reference and leaves a dipole in the difference. A cosmic ray is a spike narrower than a star. Giving the network all three at once is what lets it tell these apart. Each channel is normalised by its own noise, so a '5' means five sigma on any night. We augment training stamps with rotations and mirror images, because the sky has no 'up', and we balance training to five bogus per real, but keep validation realistic."

Slide 3: "The neural network" (1 min 30 s)

On the slide: `cnn_architecture.svg`. **Say:** "This is the network: three blocks of convolutions with 3 by 3 filters, each followed by pooling that halves the size, then a small dense head. Convolutions look for local patterns like a round blob or a sharp spike; pooling lets deeper layers see larger areas; dropout switches off 30% of neurons while training so none is relied on. It has only 72,145 weights, which is small on purpose: the stamps are small, the patterns are local, and a small model is fast on a laptop GPU and harder to overfit. We trained it with binary cross-entropy and the Adam optimiser, and we stop when validation average precision has not improved for four epochs."

Slide 4: "Results" (45 s)

On the slide: `cnn_results.svg`. **Say:** "We always compare against a simple baseline: logistic regression on six numbers. On validation nights the CNN reaches average precision 0.899 against 0.780. At the same completeness of 90%, false alarms fall from 813 per frame to 176, four and a half times fewer, and on the untouched test night the average precision is 0.938. That purity matters because it makes the next step, linking, cheap."

Slide 5: "The model found a bug in our labels" (45 s)

On the slide: `cross_fitting.svg` small, and one sentence in large type: "The most confident 'mistakes' were known asteroids." **Say:** "When we looked at the network's most confident disagreements with our labels, they were clean new point sources, and every one was a known asteroid. Our labels had missed them because the catalogue's predicted positions were about 2.9 arcseconds off, always in the same direction. We confirmed with the stars and with JPL Horizons that the error was in the catalogue, not in our images, and fixed the labels. That is data-centric ML: the model's most confident mistakes are the cheapest way to find label errors. One more design point: the next model uses this network's scores as features, so we score each training night with a network trained on the other nights, called cross-fitting, to avoid over-confident scores. Over to Satvik."

Optional whiteboard (if asked "explain backpropagation")

Draw: x -> [W1, ReLU] -> h -> [W2, sigmoid] -> p -> loss. Say: "Forward pass computes p. The loss compares p with the label y. Going backwards, the error at the output is simply p minus y for sigmoid with cross-entropy. That error times the transpose of W2, times 1 where the hidden neuron was active and 0 where it was not, gives the error at the hidden layer. Each weight's gradient is its input times the error arriving at it. Then gradient descent moves each weight a small step against its gradient. We wrote this by hand in numpy and checked it against a numerical gradient to eight decimals."

Delivery tips

- Point at the picture on slide 2: the three panels are the heart of your part.
- Say "average precision" with one sentence of meaning the first time: "a single number that summarises precision and recall for rare positives".
- Do not claim the network discovers asteroids; it removes junk blobs.

Anticipated first questions

1. "Why a CNN and not just the six numbers?" -> pixels carry shape information numbers do not; AP 0.899 vs 0.780. 2. "Why 31 x 31?" -> blur is about 2 px; 15 px of context shows edges and neighbours; small = fast. 3. "How did you avoid overfitting?" -> dropout, augmentation, early stopping, validation on other nights. 4. "Are your labels correct?" -> best available; noise sources named; two bugs found and fixed. 5. "What is cross-fitting?" -> see slide 5.

Paras: viva questions and model answers

A. Labels and data preparation

1. Where do your labels come from? Three sources: planted fake asteroids (perfect labels), known asteroids from catalogues (real labels, positions from JPL Horizons), and everything else called bogus (some label noise). The sky itself is not labelled.

2. What is injection-recovery and why do you add the fake to both science and difference images? Plant objects with known properties, run the normal pipeline, count what comes out. A real asteroid is in tonight's science image but not in the reference; adding the same stamp to science and difference keeps `reference = science − difference` unchanged, which is exactly that situation.

3. Why draw fake magnitudes uniformly instead of like real asteroids? So every brightness bin has enough fakes to measure completeness. Real asteroids are mostly faint; remember this when quoting rates, since our training mix is not the sky's mix.

4. How do you make the fakes realistic? Use ZTF's own PSF for each frame, sub-pixel rendering with exact flux, Poisson noise, the real frame times, and motion straight on the sky (converted to each frame's pixels). Then test realism with adversarial validation.

5. What is adversarial validation and what did it show? Train a classifier to separate two datasets that should look the same (fakes vs real known asteroids, magnitude-matched, cross-validated by night). AUC about 0.5 means indistinguishable; we got 0.46-0.48 (about 0.47 +/- 0.02).

6. How many chance matches do you expect when matching detections to catalogue positions? P(random point within 2 px of a detection) = detections x pi x 4 / good area = 1.15%. With 114 known asteroids in the field that is about 1.3 chance matches per frame, which is what we saw, so faint "known" matches are mostly wrong. Hence only V <= 20.5 is used.

7. What was wrong with SkyBoT positions and how did you find out? The CNN's most confident "bogus" detections were known asteroids. SkyBoT's predictions sat about 2.9 arcsec (2.7 px) from detections in a constant direction on every night. Our WCS matches Gaia to 0.04 px and JPL Horizons to 0.1-0.3 arcsec, so SkyBoT was off. We switched to Horizons positions (median error 0.26 px vs 2.72 px) with a 2-px radius.

8. Why did you pass the observatory code I41 to the catalogue query? By default positions are for the centre of the Earth; ZTF is about 6,400 km away, which shifts a 1 AU asteroid by up to about 9 arcsec, more than 4 pixels.

9. What is label noise in your "bogus" class? Variable stars, other real transients, and undiscovered asteroids are all labelled bogus. So a "false positive" on real data may be a real object, which is the thing we hunt. On a training night it is a tiny fraction of 36,000 detections, so the model barely notices.

10. Describe the stamp and why three channels. 3 x 31 x 31: science, reference, difference. A real new object shows in science and difference but not reference; a bad subtraction shows a star in both and a

dipole in difference; a cosmic ray is a spike narrower than a star. The CNN compares the three panels like a person does.

11. How do you normalise stamps and why? `(value − median) / (1.4826 x MAD)` per stamp and channel, clipped to [-10, 50]. Median and MAD are robust to a bright star; "5" then means 5 sigma on any night; clipping stops a saturated star dominating. Nothing is fitted on training data, so there is no leakage.

12. Why augment, and why only the training data? Rotations by 0/90/180/270 plus mirror give 8 versions per stamp: the sky has no up, and it acts like more data. Validation and test must measure performance on data as it really is, so they are never augmented.

13. Why balance the training set and what is the cost? With 3% positives the gradient is dominated by easy bogus examples. We use all real + 5 bogus per real (16.7% real). The cost is inflated probabilities (the model thinks real is 16.7% common), fixed afterwards by base-rate adjustment or calibration. Validation is not balanced, so precision and false-alarm numbers stay honest.

14. What went wrong with the first float16 stamps? Saturated star cores exceed 65,504, the largest float16 value, and became `inf`. Fix: clip before converting. Found by scanning every stamp file after the build, not just trusting the code.

B. Neural network theory

15. Why do you need a non-linear activation? Without it, stacked linear layers collapse into one linear function and depth adds nothing. ReLU makes each hidden neuron a "bent line"; many bent lines give a curved boundary (half-moons: logistic regression 0.88, network 0.94).

16. Explain the forward pass of a one-hidden-layer network. `z1 = X W1 + b1`, `h = ReLU(z1)`, `z2 = h W2 + b2`, `p = sigmoid(z2)`.

17. What is the loss and why does a confident wrong answer cost so much? Binary cross-entropy `−mean(y log p + (1−y) log(1−p))`. At p = 0.01 for a true positive the loss is -log 0.01 = 4.6, while a confident right answer (p = 0.99) costs 0.01.

18. Derive why dL/dz = p - y for sigmoid with cross-entropy. sigma'(z) = sigma(z)(1 - sigma(z)); the derivative of the loss with respect to p is (p - y) / (p(1 - p)); multiplying gives p - y: the sigmoid's slope cancels the log's slope.

19. What is backpropagation? The chain rule applied from the output backwards. Output weights: h^T (p - y) / n. Hidden error: (p - y) W2^T x ReLU'(z1). Hidden weights: X^T (hidden error) / n. Each layer reuses the next layer's result, so one backward sweep gives every gradient.

20. How did you check your hand-written gradients? Numerical gradient: nudge one weight by epsilon = 1e-5 on both sides, `(L(w+e) − L(w−e)) / 2e`. It matched backprop to 8 decimals (-0.02896520). Removing the ReLU' term makes the check fail at once.

21. What happens with a learning rate of 5.0, and what does Adam do? Gradient descent overshoots and diverges. Adam adapts each weight's step size using its recent gradients; a robust default.

22. Why must initial weights be random? If all weights start equal, every neuron gets the same gradient and stays identical forever.

C. The CNN

23. How many parameters does the first conv layer have? (3 x 3 x 3 + 1) x 16 = 448. In total the network has 72,145.

24. Why do spatial sizes go 31 -> 15 -> 7 -> 3? Padded 3x3 convolutions keep the size; each max-pool halves it (integer division). 64 channels x 3 x 3 = 576 numbers feed the dense layer.

25. What does dropout do? It randomly switches off 30% of the neurons in the head during training so the network cannot rely on any single neuron; it is turned off when scoring (`model.eval()`).

26. How do you know when to stop training? Validation average precision is checked every epoch; we keep the best epoch and stop after 4 epochs without improvement (the best was epoch 21, AP 0.899).

27. Why judge by validation AP and not training loss? Training loss measures memorising; only held-out nights show generalisation. Training is balanced and augmented, validation is realistic, so their losses are not even comparable (training 0.08 vs validation 0.035 can both be right).

28. Why is the CNN "better" if the completeness curves look alike at 90% recall? Because we fixed recall; the gain is purity: 176 instead of 813 false alarms per frame at the same recall. Purity makes linking cheap: only about 24% of detections remain and seed pairs drop from 52,000 to 3,500.

29. What is cross-fitting? Score each training night with a CNN trained on other nights (4 folds of 2 nights). The tracklet model uses CNN scores as features; scoring nights a CNN trained on gives over-confident scores that would not hold on new nights. Folds are by night, never by detection.

30. What mistakes did the CNN make that taught you something? Its top-scoring "bogus" detections were known asteroids our labels had missed: a label error, not a model error. On the test night, 9 tracklets counted as false positives by the tracklet model were also real known asteroids, so the effective false-positive count was 0.

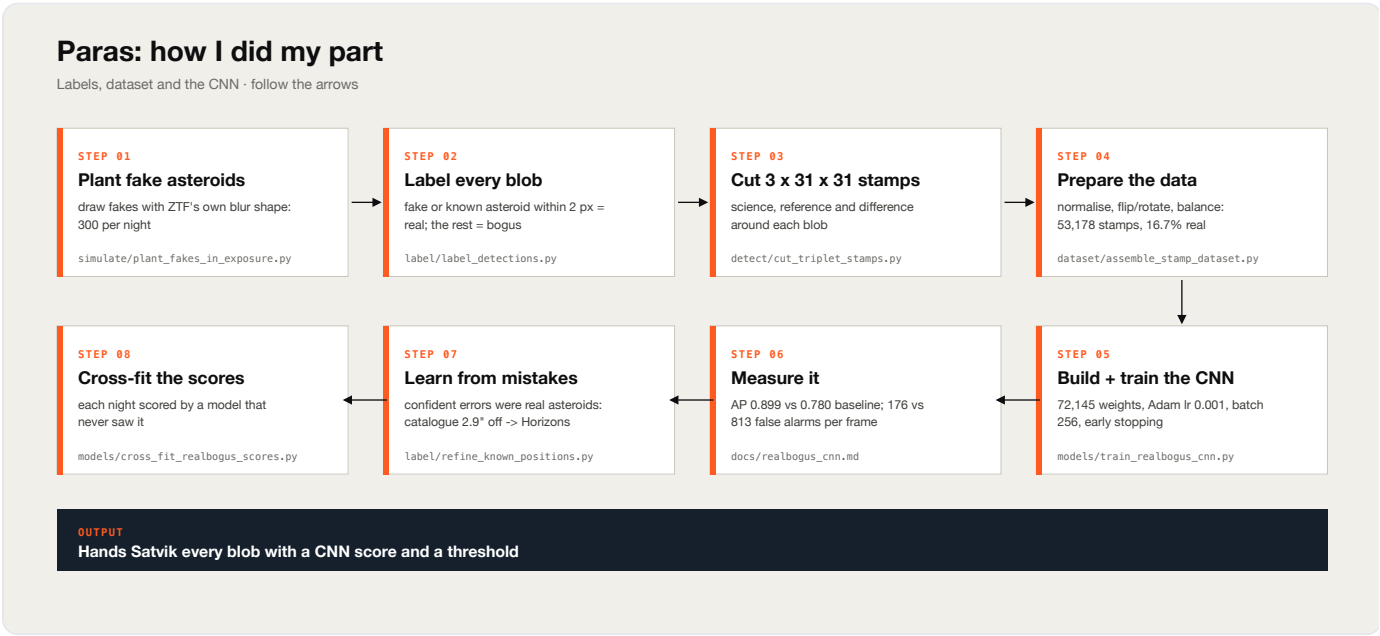
D. Judgement

31. Is 0.938 AP on the test night impressive? It beats the baseline everywhere, but AP on a set rich in planted fakes flatters; the evaluation chapter uses depth, known asteroids recovered and unexplained alarms per night.

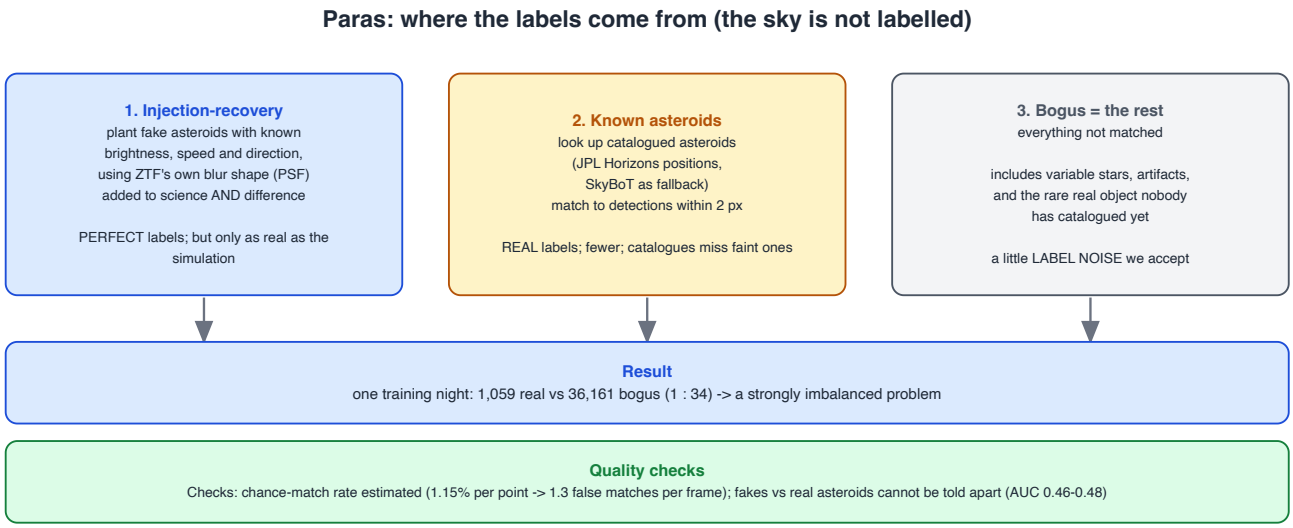
32. What would you change with more time? Human-labelled real/bogus examples to complement injections (as ZTF's own classifier uses), a bigger stamp (63 x 63), a calibration step so scores read as probabilities, and an ablation of the three channels.

33. "Did you write the model yourself?" Answer truthfully (see `00_Start_Here/07_AI_Use_and_Integrity.md`): the project was built with an AI coding assistant under Advik's direction; say what you re-ran or changed (for example the from-scratch notebook, the parameter count) and explain the network on the whiteboard without notes.

Diagrams



00 Process Diagram Paras



01 label sources

Paras: the CNN's input is three pictures of the same 31 x 31 pixels

	science	reference	difference
Real new object (asteroid)	blob	nothing	blob
Bad subtraction of a bright star	bright star	bright star	dipole / ring
Cosmic ray	sharp spike	nothing	sharp spike
Noise	nothing	nothing	faint speck
Edge of a masked area	step / stripe	step / stripe	sharp edge

Input to the CNN
3 channels x 31 x 31 pixels

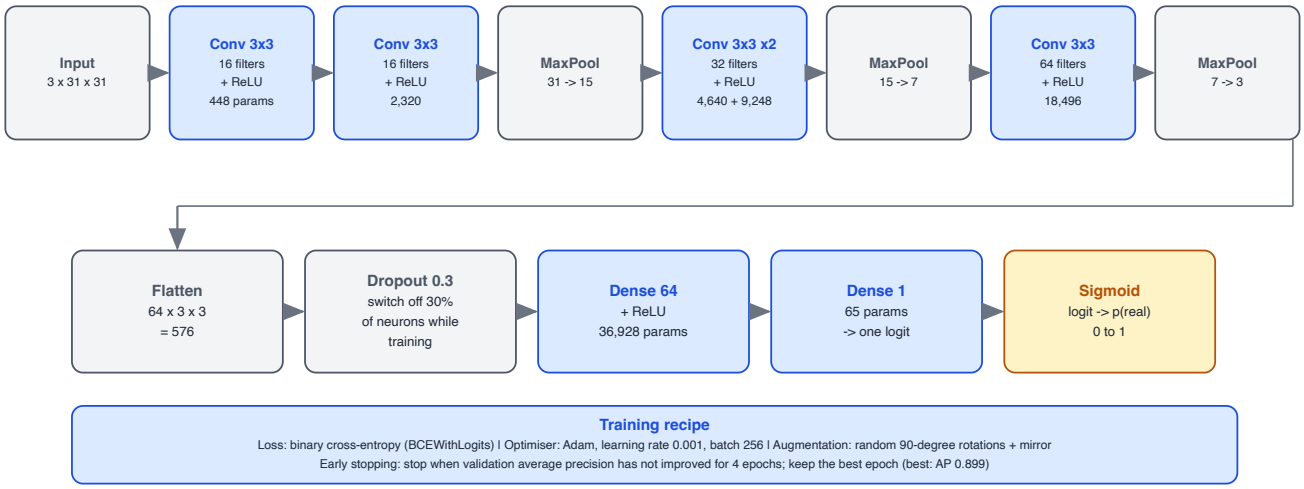
each channel normalised:
(value - median)
/ (1.4826 x MAD)
then clipped to [-10, 50]

so '5' means 5 sigma
on ANY night

A person tells these apart by comparing the three panels. The CNN learns to do the same from examples.

02 triplet stamp

Paras: the real/bogus CNN (72,145 learnable numbers)

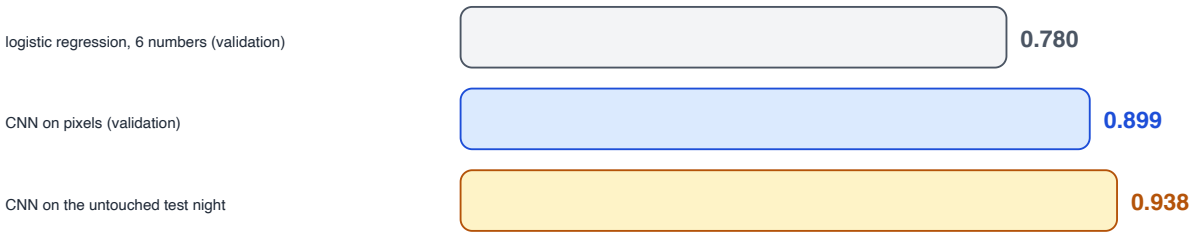


Training recipe
Loss: binary cross-entropy (BCEWithLogits) I Optimiser: Adam, learning rate 0.001, batch 256 I Augmentation: random 90-degree rotations + mirror
Early stopping: stop when validation average precision has not improved for 4 epochs; keep the best epoch (best: AP 0.899)

03 cnn architecture

Paras: CNN versus the simple baseline

Average precision (AP): one number for rare positives; 1.0 is perfect. Same 67,041 validation detections (3.2% real).



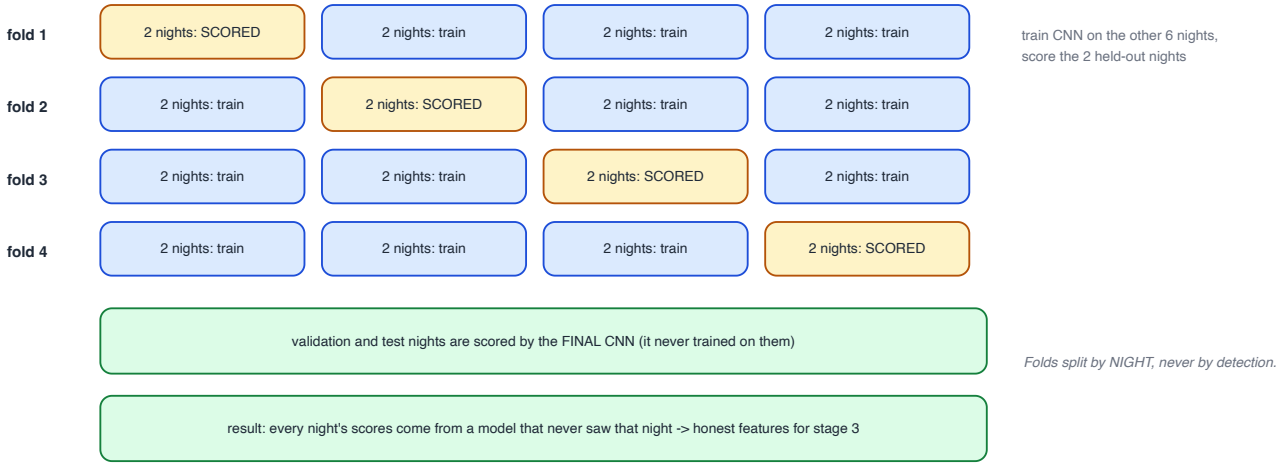
Same completeness, far fewer false alarms
at 90% recall: 813 -> 176 false alarms per frame (4.6x fewer)
precision at 90% recall: 0.191 -> 0.522

Why it matters for the next stage
the CNN's gain is PURITY: only ~24% of detections remain,
so linking seed pairs fall from 52,000 to 3,500

04 cnn results

Paras: cross-fitting, so the second model never sees over-confident scores

Problem: the tracklet model uses the CNN's scores as features. A CNN scoring its OWN training nights is over-confident.



05 cross fitting